

REDIS

Fundamentos, estruturas de dados, principais comandos e operações

Aula didática para graduação

Teoria → terminal redis-cli → exercícios → aplicações

Objetivos de aprendizagem

Ao final, o aluno deverá conseguir...

Explicar o que é Redis e quando ele é útil em uma arquitetura de software.

Identificar e escolher estruturas: String, Hash, List, Set, Sorted Set e Stream.

Executar operações CRUD e operações específicas pelo redis-cli.

Aplicar TTL, contadores, filas, ranking, Pub/Sub e transações.

Reconhecer cuidados de desempenho, memória, persistência e modelagem de chaves.

O que é Redis?

Um servidor de estruturas de dados em memória

In-memory

Os dados são mantidos principalmente em memória, favorecendo baixa latência.

Chave → valor

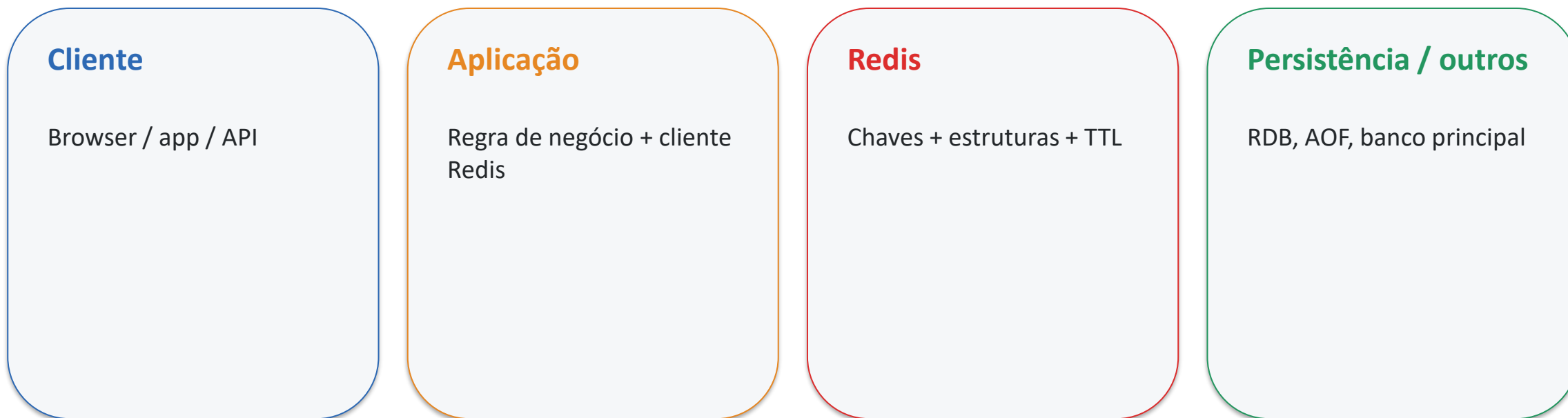
Cada objeto é localizado por uma chave; comandos operam sobre essa chave.

Além de cache

Também pode atuar em sessões, filas, contadores, eventos e outros padrões.

Redis dentro de uma aplicação

O caminho de uma requisição



Ideia-chave: Redis não precisa substituir um banco relacional; muitas arquiteturas o utilizam como componente especializado.

Como o Redis organiza os dados

A chave aponta para um valor de um tipo

```
SET aluno:1001:nome "Ana"  
HSET aluno:1001 nome "Ana" curso "Computação" periodo 3  
LPUSH fila:jobs "job-001"  
SADD turma:2026 "Ana" "Bruno"  
ZADD ranking 950 "Ana"
```

Padrão recomendado para chaves: entidade:identificador:atributo ou domínio:recurso:identificador.

Tipos de dados essenciais

A documentação oficial destaca vários tipos nativos

Strings

Texto, números, contadores, flags e dados binários.

Hashes

Objeto com campos e valores.

Lists

Sequência ordenada; útil para filas/pilhas.

Sets

Membros únicos; união, interseção e diferença.

Sorted Sets

Membros únicos associados a score; ranking.

Streams

Sequência de entradas com campos/valores; eventos.

Outros

JSON, geoespacial, probabilístico, time series, vector sets etc. conforme edição/versão.

Preparando o laboratório

redis-cli

```
# iniciar (exemplo Linux/WSL)
sudo service redis-server start
```

```
# conectar
redis-cli
```

```
# testar
PING
# PONG
```

```
# descobrir ajuda
HELP SET
HELP @string
```

A aula original usa 127.0.0.1:6379 e propõe a prática no redis-cli. [filecite:turn0file0L29-L32](#)

Comandos de inspeção e chaves

Antes de manipular dados, saiba observar o banco

```
SET produto:1 "Notebook"  
EXISTS produto:1  
TYPE produto:1  
TTL produto:1  
SCAN 0  
DEL produto:1  
DBSIZE
```

EXISTS: verifica existência. TYPE: identifica o tipo.

SCAN percorre chaves de forma incremental, preencha SCAN a KEYS em ambientes com muitas chaves.

DEL remove a chave. DBSIZE informa a quantidade de chaves da base.

Strings — CRUD e operações numéricas

A estrutura mais simples e versátil

```
SET aluno:1:nome "Carlos"  
GET aluno:1:nome  
MSET curso "ADS" campus "Centro"  
MGET curso campus  
SET contador 10  
INCR contador  
INCRBY contador 5  
DECRBY contador 2
```

Strings podem representar texto, números e contadores; INCR/INCRBY são operações atômicas no servidor.

TTL — dados com prazo de validade

Ideal para cache, sessão e tokens temporários

```
SET sessao:101 "xyz123"  
EXPIRE sessao:101 30  
TTL sessao:101  
  
# ou diretamente no SET  
SET token:abc "valor" EX 60  
  
# remover expiração  
PERSIST token:abc
```

TTL retorna o tempo restante; EXPIRE define expiração em segundos. A resolução de expiração é de 1 ms segundo a documentação atual.

Hashes — modelando objetos

Um único key pode representar vários atributos

```
HSET aluno:1001 nome "Ana" curso "Computação" periodo 3
HGET aluno:1001 nome
HMGET aluno:1001 nome curso
HGETALL aluno:1001
HEXISTS aluno:1001 curso
HINCRBY aluno:1001 periodo 1
HDEL aluno:1001 curso
```

No material-base, HSET, HGETALL e HINCRBY são usados para criação, leitura completa e contadores internos.

Lists — filas e pilhas

Elementos ordenados por posição

```
RPUSH fila:email email1 email2 email3
LRANGE fila:email 0 -1
LPOP fila:email
RPOP fila:email
LPUSH pilha:undo acao1 acao2
LRANGE pilha:undo 0 -1
```

FIFO

RPUSH + LPOP
primeiro a entrar → primeiro a sair

LIFO

LPUSH + LPOP
último a entrar → primeiro a sair

Sets — conjuntos sem duplicatas

Operações matemáticas úteis em dados

```
SADD curso:ads Ana Bruno Ana Carla  
SMEMBERS curso:ads  
SCARD curso:ads  
SISMEMBER curso:ads Bruno  
SADD curso:si Bruno Carla  
SINTER curso:ads curso:si  
SUNION curso:ads curso:si  
SDIFF curso:ads curso:si
```

Sorted Sets — ranking por score

Cada membro tem uma pontuação

```
ZADD ranking 1500 alice  
ZADD ranking 2200 bob  
ZADD ranking 1800 carla  
ZRANGE ranking 0 -1 WITHSCORES  
ZREVRANGE ranking 0 -1 WITHSCORES  
ZINCRBY ranking 800 alice  
ZRANK ranking alice
```

Aplicações: leaderboard, prioridade, pontuação, ranking e ordenação por score.

Comparando as estruturas

Escolha pela operação que o sistema precisa

String

Um valor simples
contador/cache/flag

Hash

Objeto com campos
cadastro/perfil

List

Sequência
fila/pilha

Set

Únicos
membros/interseções

Sorted Set

score + membro
ranking/prioridade

Stream

eventos ordenados
log/processamento

Regra

Modele pela consulta
que precisa ser rápida

Pub/Sub — comunicação por eventos

Publicador e assinante desacoplados

```
# Terminal 1
```

```
SUBSCRIBE noticias
```

```
# Terminal 2
```

```
PUBLISH noticias "Nova aula de Redis!"
```

```
# Terminal 1 recebe a mensagem
```

SUBSCRIBE cria um consumidor de canal; PUBLISH envia a mensagem aos assinantes conectados.

Use para notificações/eventos simples; para mensagens que precisam de histórico e processamento mais controlado, avalie Streams.

Streams — eventos persistentes e sequenciais

Uma evolução didática do conceito de mensageria

```
XADD eventos * aluno 1001 acao matricula
XADD eventos * aluno 1002 acao pagamento
XRANGE eventos - +
XLEN eventos
XREAD COUNT 10 STREAMS eventos 0
```

Streams armazenam uma sequência de entradas; cada entrada pode conter vários pares campo-valor. São úteis em pipelines de eventos.

Transações — MULTI / EXEC

Agrupe comandos para execução sequencial

```
MULTI
SET saldo:1 100
INCRBY saldo:1 50
GET saldo:1
EXEC
```

MULTI inicia o bloco; EXEC executa os comandos enfileirados.

Importante: transação Redis não é equivalente a uma transação ACID completa de banco relacional; estude WATCH quando precisar de controle otimista.

WATCH — controle otimista

Detectando alterações concorrentes

```
WATCH saldo:1  
GET saldo:1  
MULTI  
SET saldo:1 150  
EXEC
```

Se uma chave observada for alterada antes do EXEC, a transação pode ser abortada; a aplicação deve tratar a resposta e tentar novamente quando apropriado.

Pipeline — reduzir viagens pela rede

Muitos comandos podem ser enviados em lote

```
SET aluno:1 "Ana"  
SET aluno:2 "Bruno"  
SET aluno:3 "Carla"  
GET aluno:1  
GET aluno:2  
GET aluno:3
```

Pipeline é uma técnica de cliente: envia vários comandos sem esperar uma resposta a cada ida e volta.

Benefício: reduzir RTT (round-trip time). Não confunda pipeline com transação.

Operações atômicas

Por que contadores são um bom exemplo?

```
SET pageviews:home 0  
INCR pageviews:home  
INCRBY pageviews:home 5  
GET pageviews:home
```

Atomicidade

Operações individuais são processadas de forma que o comando não seja intercalado no meio de sua execução.

Uso típico

Pageviews, tentativas, rate limit, estoque e métricas.

Cache com Redis

Padrão Cache-Aside

```
# 1. tentar cache
GET produto:100

# 2. se não existir: consultar banco
# 3. guardar resultado por tempo limitado
SET produto:100 "...json..." EX 300

# 4. próximas requisições usam o cache
```

Objetivo: reduzir carga e latência do banco principal. Defina TTL, estratégia de invalidação e tamanho do cache.

Sessões e rate limiting

Dois padrões práticos

```
# sessão
SET sessao:user:1001 "token..." EX 1800

# contador por janela simples
INCR rate:user:1001
EXPIRE rate:user:1001 60
TTL rate:user:1001
```

No rate limiting real, considere cuidadosamente concorrência, janela, limites e comportamento de expiração; existem algoritmos mais robustos.

Persistência: RDB x AOF

Redis é in-memory, mas pode persistir dados

RDB

Snapshots em pontos no tempo.
Compacto e útil para backup/recovery.
Pode perder alterações desde o último snapshot.

AOF

Registra operações de escrita.
Maior durabilidade, dependendo do fsync.
Pode exigir mais espaço e recuperação.

Ambos / nenhum

É possível combinar estratégias ou desabilitar persistência em cenários como cache.

A estratégia depende da tolerância à perda de dados e dos requisitos de desempenho.

Memória e eviction

O que acontece quando o espaço fica limitado?

Redis permite políticas de eviction para decidir quais chaves podem ser removidas quando a memória chega ao limite.

Exemplos de políticas incluem LRU/LFU e variantes voltadas a chaves com TTL.

Em cache, eviction pode ser parte do desenho; em dados críticos, a estratégia deve considerar durabilidade e capacidade.

Monitore memória, hit rate, evictions e tamanho das estruturas.

Modelagem de chaves

Uma convenção simples melhora manutenção

```
usuario:1001  
usuario:1001:perfil  
usuario:1001:sessao  
produto:100:preco  
curso:2026:alunos  
ranking:programacao
```

Segurança e operação

Comandos não são apenas SET e GET

Autenticação e ACLs devem ser configuradas conforme o ambiente.

Evite expor Redis diretamente à Internet; controle rede, portas e acesso.

Tenha cuidado com comandos administrativos/destrutivos em ambientes compartilhados.

Use observabilidade: memória, CPU, latência, conexões, comandos, hits/misses e evictions.

Em produção, trate persistência, backup, replicação e recuperação como parte da arquitetura.

Exercício guiado — Cadastro de alunos

15 minutos • execute no redis-cli

```
HSET aluno:1 nome "Ana" curso "ADS" periodo 2
HSET aluno:2 nome "Bruno" curso "SI" periodo 4
HGETALL aluno:1
HGET aluno:2 curso
HINCRBY aluno:1 periodo 1
EXISTS aluno:1
TYPE aluno:1
```

Desafio: acrescente e-mail, nota e status; depois crie uma chave de sessão com TTL.

Exercício guiado — fila e ranking

15 minutos • duas estruturas em um cenário

```
RPUSH fila:trabalhos job1 job2 job3
LRANGE fila:trabalhos 0 -1
LPOP fila:trabalhos

ZADD ranking 80 ana
ZADD ranking 95 bruno
ZADD ranking 88 carla
ZREVRANGE ranking 0 -1 WITHSCORES
```

Pergunta: qual estrutura você escolheria para fila de atendimento? E para ranking de notas? Explique pela operação necessária.

Desafio final — mini sistema

Monte um pequeno domínio usando 4 estruturas

String: contador de acessos.

Hash: cadastro do usuário.

List: fila de tarefas.

Set: conjunto de cursos do aluno.

Sorted Set: ranking de pontuação.

TTL: sessão temporária.

Pub/Sub: notificação de evento.

Entregável: comandos utilizados + desenho simples da solução + justificativa das estruturas.

Checklist de comandos

Folha rápida para a aula prática

Chaves

EXISTS • TYPE • DEL • SCAN
EXPIRE • TTL • PERSIST

String

SET • GET • MGET
INCR • INCRBY • DECRBY

Hash

HSET • HGET • HMGET
HGETALL • HDEL • HINCRBY

List/Set

LPUSH • RPUSH • LRANGE • LPOP
SADD • SMEMBERS • SISMEMBER •
SINTER

Sorted/Events

ZADD • ZRANGE • ZREVRANGE •
ZINCRBY
XADD • XRANGE • XREAD

Controle

MULTI • EXEC • WATCH
SUBSCRIBE • PUBLISH

Resumo e referências

Do comando ao desenho da solução

Redis é um servidor de estruturas de dados em memória; seu valor está em escolher a estrutura adequada ao problema.

Comece dominando: String, Hash, List, Set, Sorted Set, TTL, Pub/Sub, Streams e transações.

Depois avance para persistência, replicação, observabilidade, segurança, cluster e bibliotecas de programação.

Material-base: apresentação fornecida pelo professor, que cobre fundamentos, estruturas, comandos e uma prática com 20 exercícios. [?filecite?turn0file0?L44-L50?](#)

Referência principal: documentação oficial do Redis — tipos de dados e referência de comandos.